

Redux Flow — Explained Simply

ChatWave · Real-Time Chat Application

How data moves through Redux Toolkit in your MERN chat app —
store, slices, actions, dispatch, selectors, and redux-persist.

Frontend: React + Redux Toolkit + redux-persist

1. What is Redux? (the simple idea)

Redux is a **central box** that holds data your whole app needs to share — like the logged-in user, the list of friends, the selected chat, and the messages. Instead of passing this data through many components (prop-drilling), every component reads it directly from this one box.

That central box is called the **store**. The big benefit: when the data in the store changes, every component using it **updates automatically**.

The 4 words you must know

Term	Meaning (in plain English)
Store	The single box that holds all shared data (state).
Slice	A labelled section of the store + the functions that change it.
Action / Reducer	An instruction to change the store (e.g. 'set the messages').
dispatch	How a component SENDS an instruction to change the store.

And one more for reading: **useSelector** — how a component READS data from the store.

2. The golden flow (read this twice)

Component	dispatch(action)	Reducer updates Store	useSelector re-renders UI
-----------	------------------	-----------------------	---------------------------

- A component calls **dispatch(someAction(data))** — "please change the store".
- The matching **reducer** updates that slice of the store.
- Every component using **useSelector** on that data **re-renders** with the new value.

That's the entire cycle. Everything below is just this same loop with your real data.

3. Your store setup

Your store combines **three slices** and is wrapped with **redux-persist** (explained in section 6):

```
combineReducers({
  user: userReducer,      // who am I, friends, selected chat, online users
  message: messageReducer, // the messages in the open chat
  socket: socketReducer   // the live socket connection
})
```

So your store looks like this in memory:

```
store = {
  user:    { authUser, otherUsers, selectedUser, onlineUsers },
  message: { messages },
  socket:  { socket }
}
```

4. Your three slices in detail

userSlice → store.user

State	What it holds	Action to change it
authUser	The logged-in user (you)	setAuthUser
otherUsers	List of all other users (friends)	setOtherUsers
selectedUser	The friend whose chat is open	setSelectedUser
onlineUsers	IDs of users currently online	setOnlineUsers

messageSlice → store.message

State	What it holds	Action to change it
messages	All messages in the open conversation	setMessages

socketSlice → store.socket

State	What it holds	Action to change it
socket	The live Socket.IO connection object	setSocket

5. Real examples from YOUR app

A) Logging in

When login succeeds, the component saves you into the store:

```
dispatch(setAuthUser(res.data)); // store.user.authUser = you
```

Now the whole app knows who you are — the sidebar shows your name, and HomePage stops redirecting to /login.

B) Opening a chat

Clicking a friend in the sidebar:

```
dispatch(setSelectedUser(user)); // store.user.selectedUser = that friend
```

MessageContainer reads `selectedUser` with `useSelector`, so it instantly switches to that conversation.

C) Sending a message

```
dispatch(setMessages([...messages, newMessage]));
```

The new message is added to `store.message.messages`, and the chat re-renders to show it.

D) Real-time incoming messages (Socket.IO)

Your **socket** lives in the store. When the server pushes a 'newMessage', the hook adds it:

```
socket.on("newMessage", (m) => dispatch(setMessages([...messages, m])));
```

E) Who is online

The socket emits 'getOnlineUsers'; you store the list so green dots appear:

```
dispatch(setOnlineUsers(onlineUsers)); // store.user.onlineUsers
```

6. redux-persist — staying logged in on refresh

Normally, the Redux store is wiped when you refresh the page. **redux-persist** automatically saves your store into the browser's **localStorage** and reloads it on startup — so you stay logged in and your selected chat is remembered.

- **persistReducer** wraps your reducers and saves state to storage under the key 'root'.
- **persistStore** + **PersistGate** (in index.js) restore the saved state before the app renders.
- The **socket** is excluded from saving (it's a live connection, not data) via `ignoredPaths`.

This is why, even after pressing refresh, you don't get logged out.

7. End-to-end: from login to chatting

Login	setAuthUser	Load friends	Pick a friend	Send / receive
-------	-------------	--------------	---------------	----------------

1. You log in → **dispatch(setAuthUser)** → store knows you.
2. The `useGetOtherUsers` hook fetches friends → **dispatch(setOtherUsers)** → sidebar fills up.
3. Socket connects → **dispatch(setSocket)** and online list → **dispatch(setOnlineUsers)**.
4. You click a friend → **dispatch(setSelectedUser)** → chat opens.
5. `useGetMessages` fetches history → **dispatch(setMessages)** → bubbles appear.
6. You type & send → **dispatch(setMessages)**; friend's message arrives via socket → **dispatch(setMessages)**.
7. **redux-persist** quietly saves everything so a refresh keeps you in.

Remember the one loop: a component **dispatches** an action → the **reducer** updates the **store** → components reading that data with **useSelector** re-render. Everything in this app is just that loop, repeated.